DigitalSkola ALGORYTHM

Used Car Price Predictions

GROUP 1 - ALGORYTHM

Our Team

A collaboration between 'Algorithm' and 'Rhythm,' symbolizing the harmony between algorithmic logic and the rhythm of teamwork

ALGORYTHM

DigitalSkola

Ahmad Hamzah Syafiq
- Project lead
- Project Scoping & Objective Setting
- ML Model Developer

Rayna Fiona Fridashaina
- Project Scoping & Objective Setting
- Deck Preparation

Muhammad Thariq Ziyad Al Ghazali
- Project Scoping & Objective Setting
- Deck Preparation

Alfiyah Shaldzabila Yustin
- Team Lead
- Project Scoping & Objective Setting
- Deck Preparation

Della Fransiska Lubis
- Project Scoping & Objective Setting
- Deployment & MLOps

Luthfia Nurus Sa'adah
- Project Scoping & Objective Setting
- Deployment & MLOps

DigitalSkola

ALGORYTHM

EXECUTIVE SUMMARY

1

Project Description

This project is to explore the factors influencing car resale prices.

2

Objective

- Predict the resale price of used cars.
- Develop a platform that potential buyers/seller can easily access.
- Identify the key features that most influence the resale price.

DigitalSkola

ALGORYTHM

Data Overview

	make_year	mileage_kmpl	engine_cc	fuel_type	owner_count	price_usd	brand	transmission	color	service_history	accidents_reported	insurance_valid
0	2001	8.17	4000	Petrol	4	8587.64	Chevrolet	Manual	White	NaN	0	No
1	2014	17.59	1500	Petrol	4	5943.50	Honda	Manual	Black	NaN	0	Yes
2	2023	18.09	2500	Diesel	5	9273.58	BMW	Automatic	Black	Full	1	Yes
3	2009	11.28	800	Petrol	1	6836.24	Hyundai	Manual	Blue	Full	0	Yes
4	2005	12.23	1000	Petrol	2	4625.79	Nissan	Automatic	Red	Full	0	Yes

This dataset contains 12 columns and 10,000 entries, consisting of both numerical and categorical features relevant for predicting used car prices.

DigitalSkola

Type Data Overview

ALGORYTHM

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   make_year           10000 non-null  int64
 1   mileage_kmpl        10000 non-null  float64
 2   engine_cc           10000 non-null  int64
 3   fuel_type           10000 non-null  object
 4   owner_count         10000 non-null  int64
 5   price_usd           10000 non-null  float64
 6   brand               10000 non-null  object
 7   transmission        10000 non-null  object
 8   color               10000 non-null  object
 9   service_history     7962 non-null   object
 10  accidents_reported  10000 non-null  int64
 11  insurance_valid     10000 non-null  object
dtypes: float64(2), int64(4), object(6)
memory usage: 937.6+ KB
```

Most columns are complete, but the service_history column has missing values that require data cleaning before analysis or model building.

DigitalSkola

ALGORYTHM

Data Cleaning

Filling Missing Values

```
car_dataset_2['service_history'] = car_dataset_2['service_history'].fillna('No Service')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   make_year           10000 non-null  int64
 1   mileage_kmpl        10000 non-null  float64
 2   engine_cc           10000 non-null  int64
 3   fuel_type           10000 non-null  object
 4   owner_count         10000 non-null  int64
 5   price_usd           10000 non-null  float64
 6   brand               10000 non-null  object
 7   transmission        10000 non-null  object
 8   color               10000 non-null  object
 9   service_history     10000 non-null  object
 10  accidents_reported  10000 non-null  int64
 11  insurance_valid     10000 non-null  object
dtypes: float64(2), int64(4), object(6)
memory usage: 937.6+ KB
```

Check Duplicate Data

```
car_dataset_2.duplicated().sum()
```

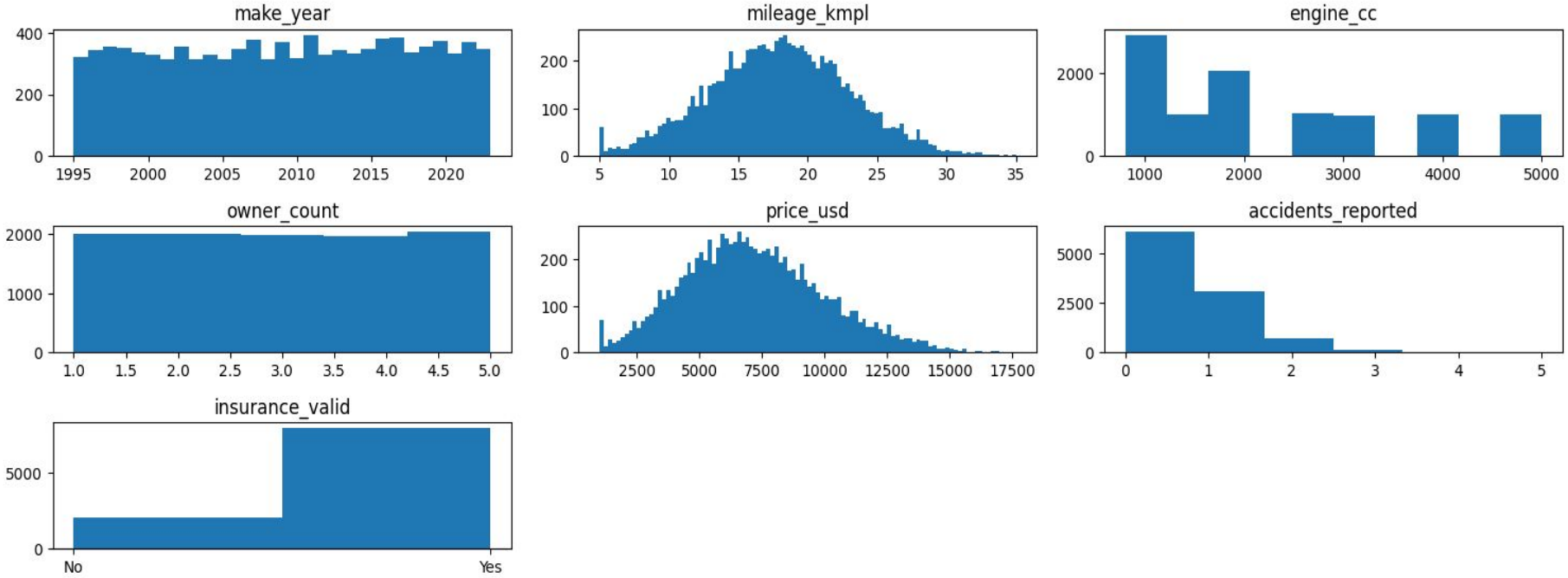
```
np.int64(0)
```

DigitalSkola

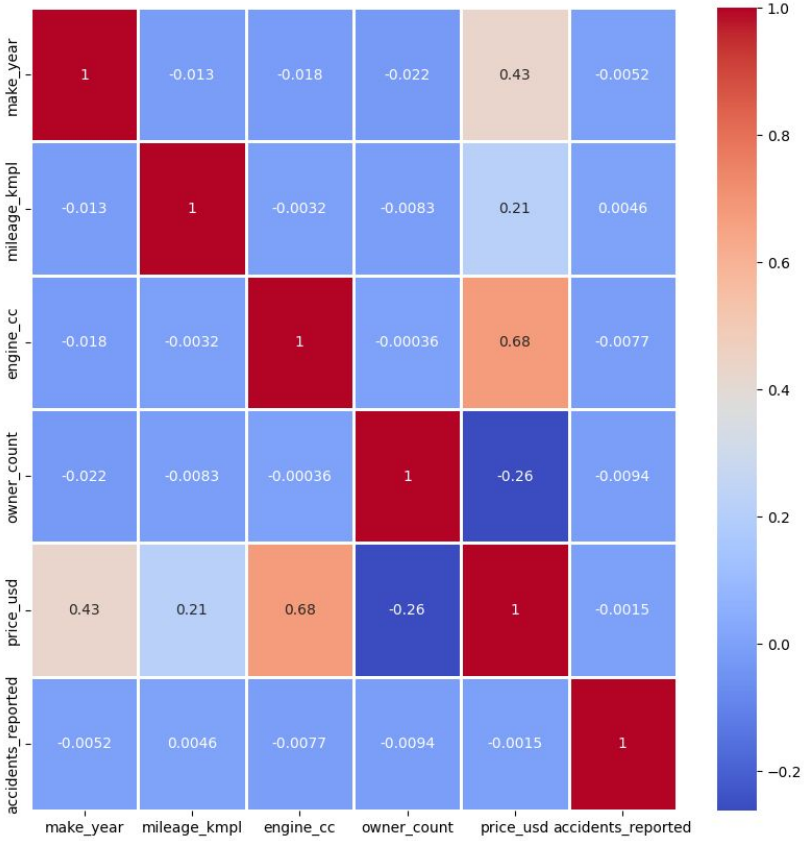
Exploratory Data Analysis

ALGORYTHM

Histograms of Numerical Columns



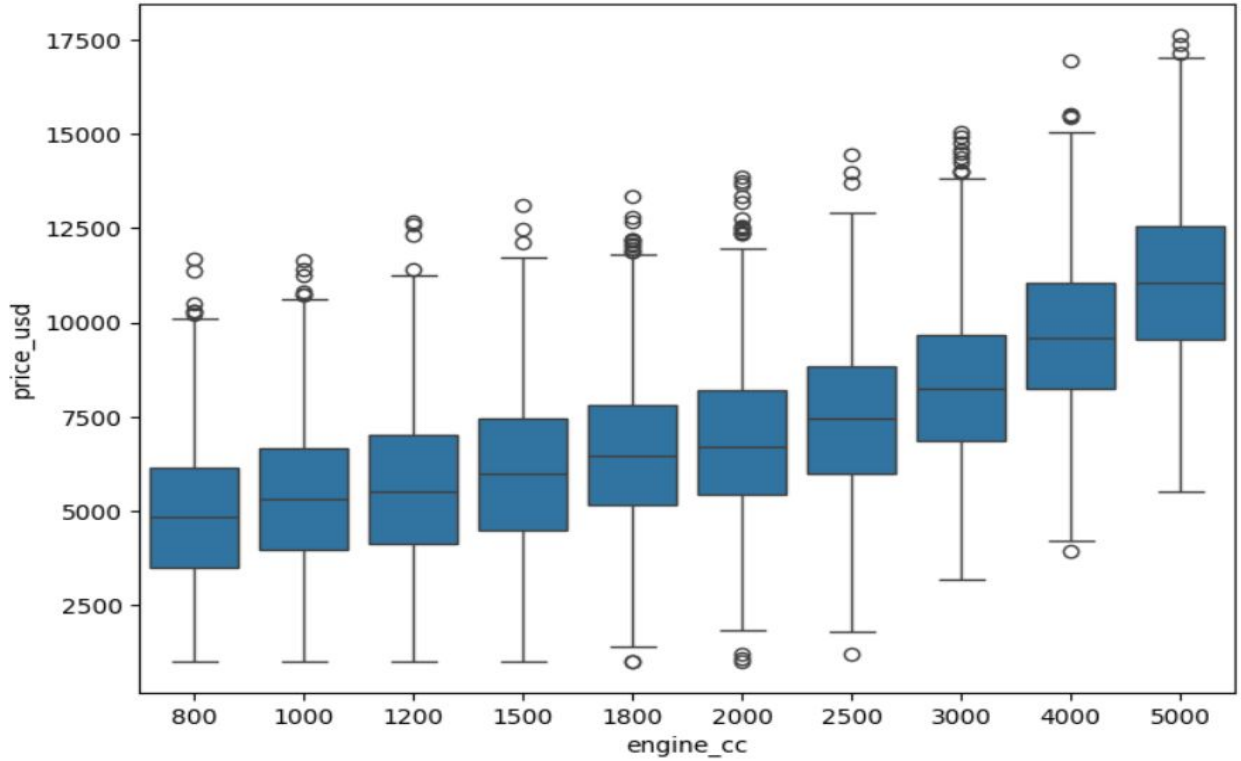
Exploratory Data Analysis



Based on the heatmap mapping, it is known that the features that have the greatest influence on the car's resale price are:
Engine CC (0.68)
Make Year (0.43)
Mileage (0.21)
Owner Count (−0.26)
Accident Reported (−0.0015)

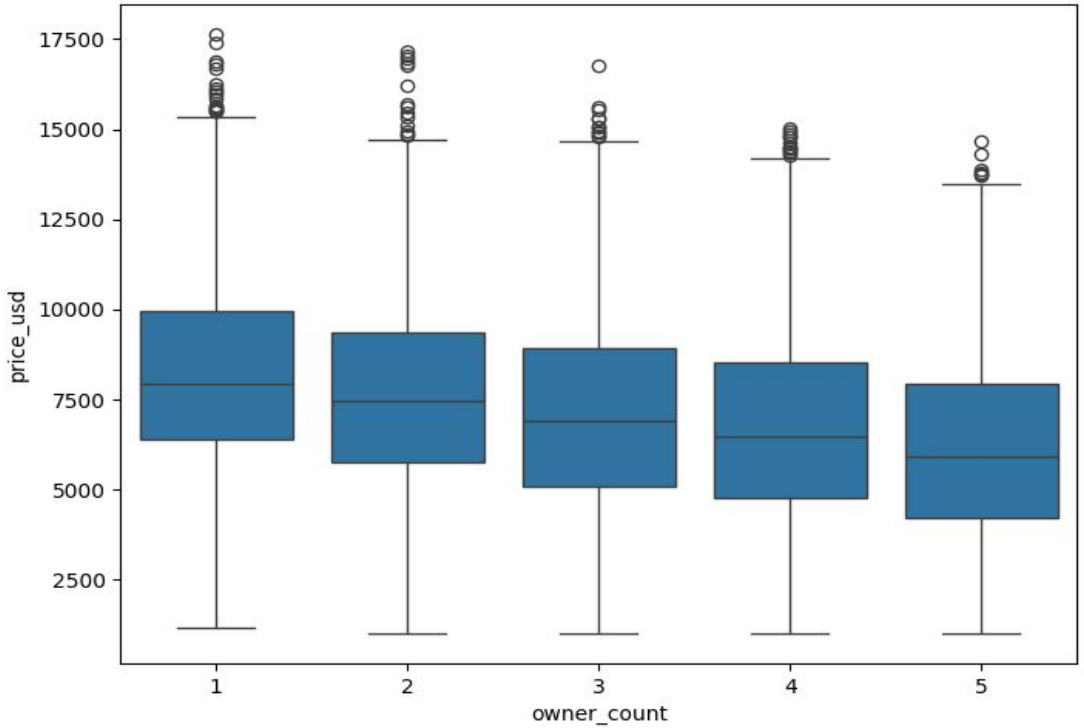
Exploratory Data Analysis

This chart shows that the larger the engine CC of a car, the higher its price tends to be.



Exploratory Data Analysis

This chart shows that the greater the number of previous owners, the lower the car's price tends to be.



Data Preprocessing

Label Encoding

Split Dataset into Feature and Target

Split Dataset into Train and Test

Feature Scaling

Label Encoding

```
encoders = {}

label_encoders_count = 0
for col in car_dataset_3.columns[:]:
    if car_dataset_3[col].dtype == 'object':
        if len(list(car_dataset_3[col].unique())) <= 2:
            print(col)
            label_encoders = LabelEncoder()
            label_encoders.fit(car_dataset_3[col])
            car_dataset_3[col] = label_encoders.transform(car_dataset_3[col])
            encoders[col] = label_encoders
            label_encoders_count += 1
print('{} columns were label encoded.'.format(label_encoders_count))
```

Binary Label Encoding

```
transmission
insurance_valid
2 columns were label encoded.
```

```
car_dataset_3 = pd.get_dummies(car_dataset_3)
```

One Hot Encoding

DigitalSkola

ALGORYTHM

Label Encoding

Binary Label Encoding

Before

insurance_valid
No
Yes
Yes
Yes
Yes
...
Yes

After

insurance_valid
0
1
1
1
1
...
1

One Hot Encoding

Before

fuel_type
Petrol
Petrol
Diesel
Petrol
Petrol
...
Petrol

After

fuel_type_Diesel	fuel_type_Electric	fuel_type_Petrol
False	False	True
False	False	True
True	False	False
False	False	True
False	False	True
...	...	...
False	False	True
True	False	False

DigitalSkola

ALGORYTHM

Split Dataset

Into Feature and Target

```
target_dataset = car_dataset_3["price_usd"]
feature_dataset = car_dataset_3.drop(columns="price_usd")
```

In this dataset, the target variable is price_usd, and all the other columns are used as features.

Into Train and Test

```
x_train, x_test, y_train, y_test = train_test_split(feature_dataset, target_dataset,
                                                    test_size = 0.2,
                                                    random_state = 0)
```

This dataset is split into 80% training and 20% testing.
The random_state is used to ensure that the data split remains consistent every time.

DigitalSkola

ALGORYTHM

Feature Scaling

StandardScaler is used to standardize all numerical features. The fitting process is applied on the training data, and the transformation is then applied to the testing data.

```
scaler_x = StandardScaler()
x_train2 = pd.DataFrame(scaler_x.fit_transform(x_train))
x_train2.columns = x_train.columns.values
x_train2.index = x_train.index.values
x_train = x_train2

x_test2 = pd.DataFrame(scaler_x.transform(x_test))
x_test2.columns = x_test.columns.values
x_test2.index = x_test.index.values
x_test = x_test2
```

DigitalSkola

ALGORYTHM

Model Selections

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.metrics import make_scorer, r2_score

def test_model(model, X_train=x_train, y_train=y_train):
    cv = KFold(n_splits = 10, shuffle=True, random_state = 45)
    r2 = make_scorer(r2_score)

    r2_val_score = cross_val_score(model, x_train, y_train, cv=cv, scoring = r2)
    score = [r2_val_score.mean()]
    return score
```

The training dataset is divided into 10 folds. With this approach, the model is trained and validated 10 times, resulting more stable evaluation

Model Selections

1. Decision Tree
2. Random Forest
3. Linear
4. XG Boost
5. Support Vector
6. Lasso
7. Ridge
8. ElasticNet
9. Polynomial

Output:
Average R^2 value → the higher the better.

Regression Model	R^2 Score
Decision Tree	0.7008
Random Forest	0.8472
Linear	0.87006
XG Boost	0.8413
Support Vector	0.8481
Lasso	0.8701
Ridge	0.8701
ElasticNet	0.7717
Polynomial	0.8649

Model Selections

The best performance is Lasso and Ridge with R^2 of 0.8701.

DigitalSkola ALGORYTHM

Hyperparameter Tuning

Lasso and Ridge model was chosen as the best model so far. the hyperparameter tuning required to maximize the model performance. These steps consist of:

- Finding Optimal Alpha for Lasso Regression
- Finding Optimal Alpha for Ridge Regression
- KFold parameter tuning (including data train and test splits)

DigitalSkola ALGORYTHM

Hyperparameter Tuning

- Finding Optimal Alpha for Lasso Regression:

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.metrics import make_scorer, r2_score
from sklearn.linear_model import Lasso

def test_model(model, X_train, y_train):
    cv = KFold(n_splits = 10, shuffle=True, random_state = 45)
    r2 = make_scorer(r2_score)

    r2_val_score = cross_val_score(model, X_train, y_train, cv=cv, scoring = r2)
    score = [r2_val_score.mean()]
    return score

def find_optimal_lasso_alpha(alphas, X_train, y_train, test_model_func):
    best_alpha = None
    best_r2_score = -np.inf

    for alpha in alphas:
        # Increased max_iter further to help with convergence
        lasso_model = Lasso(alpha=alpha, random_state=0, max_iter=1000, tol=1e-4)
        r2_score_mean = test_model_func(lasso_model, X_train, y_train)[0]

        if r2_score_mean > best_r2_score:
            best_r2_score = r2_score_mean
            best_alpha = alpha

    return best_alpha, best_r2_score

# Example usage:
alphas_to_test = np.logspace(-2, 2, 100) # Test alphas from 0.01 to 100

optimal_alpha, max_r2 = find_optimal_lasso_alpha(alphas_to_test, x_train, y_train, test_model)

print(f"Optimal Alpha for Lasso Regression: {optimal_alpha:.4f}")
print(f"Max R² score with optimal Alpha: {max_r2:.4f}")
```

```
Optimal Alpha for Lasso Regression: 11.7681
Max R² score with optimal Alpha: 0.8702
```

The optimal alpha was determined by testing alpha values from 0.01 to 100 and selecting the one that produced the best R^2 score

The final result for optimal alpha for lasso regression is alpha = 11.7681

DigitalSkola ALGORYTHM

Hyperparameter Tuning

- Finding Optimal Alpha for Ridge Regression:

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.metrics import make_scorer, r2_score

def test_model(model, X_train, y_train):
    cv = KFold(n_splits = 10, shuffle=True, random_state = 45)
    r2 = make_scorer(r2_score)

    r2_val_score = cross_val_score(model, X_train, y_train, cv=cv, scoring = r2)
    score = [r2_val_score.mean()]
    return score

def find_optimal_ridge_alpha(alphas, X_train, y_train, test_model_func):
    best_alpha = None
    best_r2_score = -np.inf

    for alpha in alphas:
        ridge_model = Ridge(alpha=alpha, random_state=0)
        r2_score_mean = test_model_func(ridge_model, X_train, y_train)[0]

        if r2_score_mean > best_r2_score:
            best_r2_score = r2_score_mean
            best_alpha = alpha

    return best_alpha, best_r2_score

# Example usage:
alphas_to_test = np.logspace(-3, 3, 100) # Test alphas from 0.001 to 1000

optimal_alpha, max_r2 = find_optimal_ridge_alpha(alphas_to_test, x_train, y_train, test_model)

print(f"Optimal Alpha for Ridge Regression: {optimal_alpha:.4f}")
print(f"Max R² score with optimal Alpha: {max_r2:.4f}")
```

```
Optimal Alpha for Ridge Regression: 2.8480
Max R² score with optimal Alpha: 0.8701
```

The optimal alpha was determined by testing alpha values from 0.01 to 100 and selecting the one that produced the best R^2 score

The final result for optimal alpha for Ridge regression is alpha = 2.8480

DigitalSkola ALGORYTHM

Hyperparameter Tuning

- KFold parameter tuning (including data train and test splits)

```
x_train, x_test, y_train, y_test = train_test_split = train_test_split(
    feature_dataset, target_dataset, test_size=0.2, random_state=42, shuffle=True)

kf = KFold(n_splits=5, shuffle=True, random_state=42)

lasso = Lasso(alpha=11.7681)
ridge = Ridge(alpha=2.8480)
```

The fine tuned KFold set to 5 splits and the fine tuned models (Lasso and Ridge) used the optimum alpha

DigitalSkola ALGORYTHM

Model Evaluation

Metric	Lasso Regression	Ridge Regression
Train R^2	0.87045	0.87093
Test R^2	0.87637	0.87654
Train MSE	1,007,333.75	1,003,578.10
Test MSE	983,580.05	982,170.08
Train RMSE	1003.66	1001.79
Test RMSE	991.76	991.04
Train MAE	798.68	797.11
Test MAE	792.31	791.03
K-Fold CV (MSE)	1,004,229.10	1,004,988.28

Both models show very similar performance — stable and not overfitting.

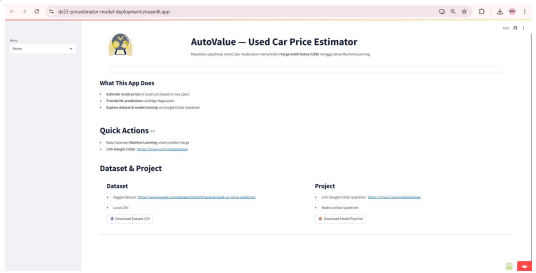
Ridge performs slightly better across almost all metrics:

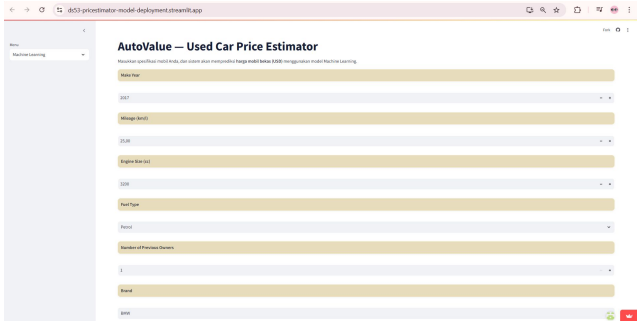
- Higher R^2
- Lower MSE, RMSE, and MAE
- The differences are small but consistent

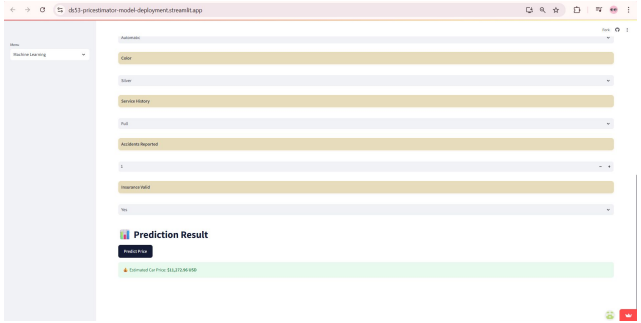
Ridge is more stable when features are correlated — making it the better choice as the final model.

Model Deployment

Model Delployed using streamlit :
https://ds53-pricestimator-model-deployment.streamlit.app/

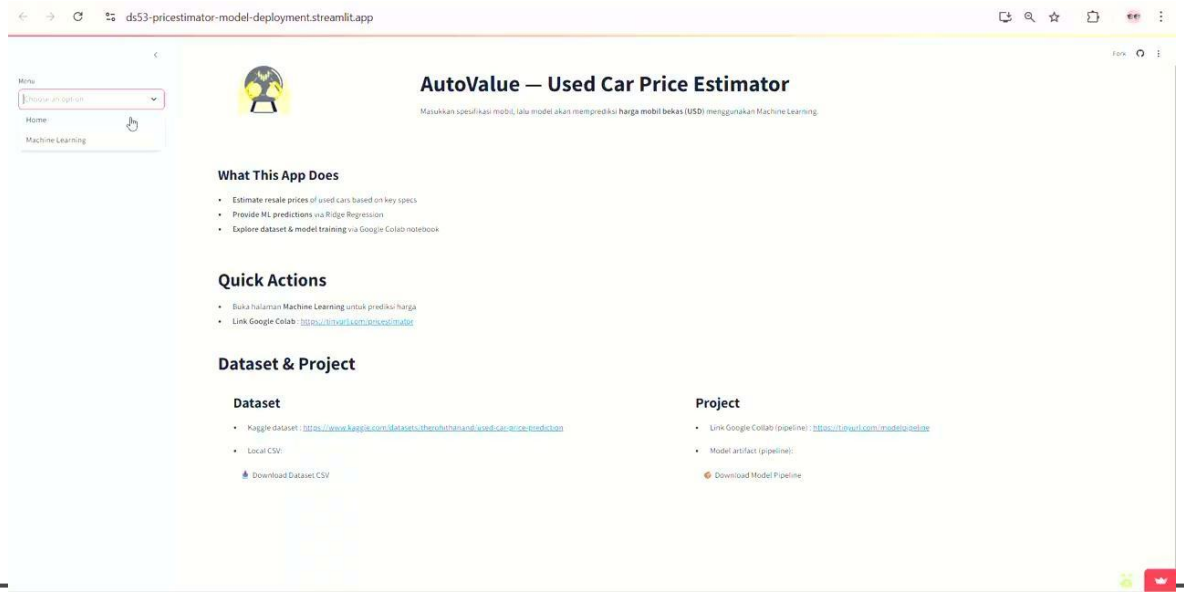






Model Deployment

Model Delployed using streamlit :
https://ds53-pricestimator-model-deployment.streamlit.app/



Conclusion & Recommendations

Conclusion

1

- Accurately predicts the resale price of used cars using a machine learning model.
- Provides an accessible platform that allows potential buyers to check estimated resale values.
- Identifies the key features influencing resale price, including engine capacity, make year, mileage, owner count, and accident history.

Recommendations

2

- Consider adding additional features or datasets that may influence car prices, such as total mileage, tax history, or categorical attributes like vehicle features (e.g., autopilot).
- Explore alternative regression models or use an ensemble of multiple regression models to improve prediction performance.

DigitalSkola ALGORYTHM

Thank You

Dataset

Notebook

Deployment